CompletableFuture, ForkJoinPool and Virtual Threads in Java

Java provides several concurrency tools, but each one solves a different problem:

- `Future` represents the eventual completion of one asynchronous task.
- `CompletableFuture` allows asynchronous operations to be chained, transformed and combined.
- `ForkJoinPool` is designed for divide-and-conquer computations.
- Virtual threads make thread-per-task programming practical for large numbers of blocking operations.

Understanding the distinction between these tools is more important than memorising their methods.

Part 1: Revisiting `Future`

1. What Does a `Future` Represent?

A `Future<T>` represents the pending result of an asynchronous task.

```
Task submitted now
        ↓
Task executes separately
        ↓
Result may become available later
        ↓
Future represents that pending result
```

Example:

```
Future<Integer> future = executor.submit(() -> {
    return 50;
});
```

The task may still be running when `submit()` returns. The returned `Future` gives the caller a way to:

- wait for the result
- check whether the task has completed
- cancel the task
- observe an exception raised by the task

---

2. The Three Common Forms of `submit()`

2.1 Submit a `Callable`

Use a `Callable` when the task must return a value.

```
Future<Integer> future = executor.submit(() -> {
    return 50;
});
```

The value can later be retrieved using:

```
Integer result = future.get();
```

2.2 Submit a `Runnable`

A `Runnable` does not return a value, but `submit()` still returns a `Future`.

```
Future<?> future = executor.submit(() -> {
    System.out.println("Saving data...");
});
```

The `Future` can be used to track:

- completion
- cancellation
- failure

After successful completion:

```
Object result = future.get();
```

`result` will be `null`.

2.3 Submit a `Runnable` With a Predefined Result

A result can be supplied separately while submitting a `Runnable`.

```
Future<String> future = executor.submit(
        () -> System.out.println("Task running"),
        "SUCCESS"
);
```

After the task completes successfully:

```
System.out.println(future.get());
```

Output:

```
SUCCESS
```

The `Runnable` itself still returns nothing. The executor returns the predefined value after successful completion.

---

3. Calling `get()` Immediately

Consider the following code:

```
Future<Integer> future = executor.submit(task);
Integer result = future.get();
```

The task is submitted asynchronously, but the calling thread immediately waits for it.

At that point, the code behaves almost like synchronous code from the caller's perspective.

A more useful pattern is:

```
Future<Integer> future = executor.submit(task);

// Perform independent work here
System.out.println("Main thread is doing other work");

Integer result = future.get();
```

The benefit comes from allowing useful work to continue before the result is actually required.

Core Methods of `Future`

4. `get()`

```
Future<Integer> future = executor.submit(() -> {
    Thread.sleep(2000);
    return 42;
});

System.out.println("Before get()");

int result = future.get();
```

```
System.out.println("After get(): " + result);
```

`get()` is a blocking operation.

```
Result ready
    → return immediately

Result not ready
    → calling thread waits
```

Exceptions From `get()`

`InterruptedException`

The thread waiting inside `get()` was interrupted.

`ExecutionException`

The asynchronous task failed.

The original exception is available through:

```
exception.getCause();
```

`CancellationException`

The task was cancelled before its result could be returned.

---

5. `get(timeout, unit)`

```
Integer result =
        future.get(2, TimeUnit.SECONDS);
```

This waits only for the specified duration.

```
Task completes within two seconds
    → return result

Task does not complete in time
    → throw TimeoutException
```

A timeout prevents the calling thread from waiting indefinitely.

---

6. `isDone()`

```java
boolean completed = future.isDone();
```

`isDone()` is non-blocking.

It reports whether the task has reached a terminal state.

That terminal state may be:

- successful completion
- exceptional completion
- cancellation

Therefore:

```
isDone() == true
```

does not necessarily mean that the task succeeded.

Example:

```java
Future<Integer> future = executor.submit(() -> {
    throw new RuntimeException("Boom");
});
```

After the task finishes:

```
future.isDone(); // true
```

But:

```
future.get(); // throws ExecutionException
```

---

7. `cancel(boolean mayInterruptIfRunning)`

```
boolean cancelled = future.cancel(true);
```

Cancellation is a request, not a guaranteed forceful termination.

Task Has Not Started

If the task is still waiting, cancellation can prevent it from starting.

Task Is Already Running

The argument controls whether interruption may be attempted.

```
future.cancel(true);
```

The executing thread may be interrupted.

```
future.cancel(false);
```

The task is not interrupted merely because of this cancellation request.

For interruption to stop a running task, the task must cooperate.

```
Runnable task = () -> {
    while (!Thread.currentThread().isInterrupted()) {
        // Perform work
    }
};
```

---

8. `isCancelled()`

```java
boolean cancelled = future.isCancelled();
```

This reports whether the task was cancelled before completing normally.

---

9. `isDone()` vs `get()`

Method	Behaviour
`isDone()`	Performs a non-blocking status check
`get()`	Waits when the result is not ready
`isCancelled()`	Checks whether cancellation occurred

```
isDone()
    → Has execution finished in any way?

get()
    → Give me the result, or report failure/cancellation
```

Part 2: CompletableFuture

10. Limitation of the Basic `Future` API

A traditional `Future` mainly allows the caller to:

- wait for completion
- poll the completion state
- cancel the task
- retrieve the result

It does not directly provide a fluent way to declare:

```
When this task completes,
transform its result,
```

```
run another operation,
combine it with another result,
and handle failure.
```

This often leads to blocking calls such as:

```
Integer result = future.get();
```

`CompletableFuture` provides a richer model based on completion stages.

---

11. What Is `CompletableFuture`?

`CompletableFuture<T>` implements both:

```
Future<T>
CompletionStage<T>
```

This gives it two roles:

```
Future
    → represents a pending result

CompletionStage
    → represents one stage in a larger asynchronous pipeline
```

The result of one stage can automatically trigger another stage.

```
Fetch data
    ↓
Transform data
    ↓
Validate data
    ↓
Save data
    ↓
Send response
```

The caller does not have to block between every step.

---

12. Creating a CompletableFuture

12.1 `runAsync()`

Use `runAsync()` when the task does not return a value.

```
CompletableFuture<Void> future =
        CompletableFuture.runAsync(() -> {
            System.out.println("Task running");
        });
```

Its result type is:

```
CompletableFuture<Void>
```

12.2 `supplyAsync()`

Use `supplyAsync()` when the task returns a value.

```
CompletableFuture<Integer> future =
        CompletableFuture.supplyAsync(() -> {
            return 10;
        });
```

Simplified form:

```
CompletableFuture<Integer> future =
        CompletableFuture.supplyAsync(() -> 10);
```

Comparison

Method	Functional interface	Result
`runAsync()`	`Runnable`	No value
`supplyAsync()`	`Supplier<T>`	Returns a value

---

13. Default Executor

When no executor is supplied, asynchronous CompletableFuture methods normally use:

```
ForkJoinPool.commonPool()
```

Example:

```
CompletableFuture<Integer> future =
        CompletableFuture.supplyAsync(() -> 10);
```

A custom executor can be supplied explicitly:

```
ExecutorService executor =
        Executors.newFixedThreadPool(4);

CompletableFuture<Integer> future =
        CompletableFuture.supplyAsync(
                () -> 10,
                executor
        );
```

Using a custom executor is useful when:

- the common pool should not be shared with this workload
- the application needs a specific concurrency limit
- tasks perform blocking I/O
- thread naming or monitoring is required
- different task categories require isolation

Chaining Completion Stages

14. `thenApply()` : Transform a Result

`thenApply()` receives the previous result and returns a transformed value.

```
CompletableFuture<Integer> future =
        CompletableFuture
                .supplyAsync(() -> 10)
                .thenApply(value -> value * 2);
```

Flow:

```
Produce 10
    ↓
Multiply by 2
    ↓
Produce 20
```

Type transformation is also possible:

```
CompletableFuture<String> future =
        CompletableFuture
                .supplyAsync(() -> 10)
                .thenApply(value -> "Result: " + value);
```

```
CompletableFuture<Integer>
        ↓
CompletableFuture<String>
```

---

15. `thenAccept()` : Consume a Result

`thenAccept()` receives the previous result but does not produce another value.

```
CompletableFuture<Void> future =
        CompletableFuture
                .supplyAsync(() -> 10)
                .thenAccept(result -> {
```

```
                    System.out.println(result);
                });
```

It behaves like a `Consumer<T>`.

```
Receive value
    ↓
Perform side effect
    ↓
No new result
```

Typical uses include:

- printing a value
- saving a value
- sending a notification
- writing to an external system

---

16. `thenRun()`: Run the Next Action

`thenRun()` does not receive the previous result and does not return a new value.

```
CompletableFuture<Void> future =
        CompletableFuture
                .supplyAsync(() -> 10)
                .thenRun(() -> {
                    System.out.println("Pipeline completed");
                });
```

Use it when only completion matters.

---

17. Chaining Methods Compared

Method	Receives previous result?	Produces a new result?	Main purpose
`thenApply()`	Yes	Yes	Transform
`thenAccept()`	Yes	No	Consume
`thenRun()`	No	No	Run after completion

Non-Async vs Async Continuations

18. `thenApply()` and `thenApplyAsync()`

```
future.thenApply(value -> value * 2);
```

A non-`Async` continuation does not promise a new thread.

It may run:

- in the thread that completes the previous stage, or
- in the thread attaching the continuation if the stage is already complete

Therefore, this description is incorrect:

```
thenApply() always runs in the same thread
```

The actual rule is:

> A non-`Async` stage may run in a thread involved in completing or registering the stage.

Async Version

```
future.thenApplyAsync(value -> value * 2);
```

The async version schedules the continuation using the stage's default asynchronous executor.

For a normal `CompletableFuture`, this is usually the common ForkJoin pool.

A specific executor can also be provided:

```
future.thenApplyAsync(
        value -> value * 2,
        executor
);
```

Comparison

Version	Execution policy
`thenApply()`	May execute in a thread involved in completion
`thenApplyAsync()`	Scheduled using the default async executor
`thenApplyAsync(fn, executor)`	Scheduled using the supplied executor

The same pattern exists for several methods:

```
thenAccept()
thenAcceptAsync()

thenRun()
thenRunAsync()
```

Combining CompletableFutures

19. `thenCombine()`

`thenCombine()` combines the successful results of two independent futures.

```
CompletableFuture<Integer> first =
        CompletableFuture.supplyAsync(() -> 10);
```

```
CompletableFuture<Integer> second =
        CompletableFuture.supplyAsync(() -> 20);

CompletableFuture<Integer> combined =
        first.thenCombine(
                second,
                (left, right) -> left + right
        );
```

Result:

```
30
```

Flow:

```
First future  ──→ 10 ──┐
                       ├── combine → 30
Second future ──→ 20 ──┘
```

The combining function runs after both stages complete normally.

---

20. Real-World Combination Example

Suppose user details and order details can be loaded independently.

```
CompletableFuture<String> userFuture =
        CompletableFuture.supplyAsync(
                () -> fetchUser()
        );

CompletableFuture<String> orderFuture =
        CompletableFuture.supplyAsync(
                () -> fetchOrder()
        );
```

```
CompletableFuture<String> responseFuture =
        userFuture.thenCombine(
                orderFuture,
                (user, order) ->
                        user + " | " + order
        );
```

The two independent operations can run concurrently.

Handling Completion and Failure

21. `exceptionally()`

`exceptionally()` provides a recovery value when a previous stage fails.

```
CompletableFuture<Integer> future =
        CompletableFuture
                .supplyAsync(() -> {
                    throw new RuntimeException(
                            "Service unavailable"
                    );
                })
                .exceptionally(exception -> {
                    System.out.println(
                            exception.getMessage()
                    );

                    return 0;
                });
```

It is similar to a fallback operation.

---

22. `whenComplete()`

`whenComplete()` observes success or failure without normally changing the result.

```
CompletableFuture<Integer> future =
        CompletableFuture
                .supplyAsync(() -> 10)
                .whenComplete((result, exception) -> {
                    if (exception == null) {
                        System.out.println(
                                "Completed: " + result
                        );
                    } else {
                        System.out.println(
                                "Failed: " +
                                exception.getMessage()
                        );
                    }
                });
```

Use it for:

- logging
- metrics
- cleanup
- observing the final outcome

---

23. `handle()`

`handle()` receives both the result and the exception and can produce a new result in either case.

```
CompletableFuture<String> future =
        CompletableFuture
                .supplyAsync(() -> 10 / 0)
                .handle((result, exception) -> {
                    if (exception != null) {
```

```
                    return "Fallback response";
                }

                return "Result: " + result;
            });
```


Comparison

Method	Runs on success	Runs on failure	Can transform result
`exceptionally()`	No	Yes	Yes
`whenComplete()`	Yes	Yes	Normally observes only
`handle()`	Yes	Yes	Yes

Blocking and Non-Blocking Composition

24. `get()` and `join()`

A `CompletableFuture` can still be waited on.

```
Integer result = future.get();
```

It also provides:

```
Integer result = future.join();
```

Both wait when the result is not ready.

The main difference is exception handling:

Method	Failure wrapper
`get()`	`ExecutionException`
`join()`	`CompletionException`

`get()` also requires checked-exception handling.

```
try {
    Integer result = future.get();
} catch (InterruptedException exception) {
    Thread.currentThread().interrupt();
} catch (ExecutionException exception) {
    System.out.println(exception.getCause());
}
```

`join()` uses unchecked exceptions:

```
Integer result = future.join();
```

---

25. Avoid Blocking Between Every Stage

This defeats much of the purpose of composition:

```
Integer firstResult = firstFuture.get();
Integer secondResult = secondFuture.get();

Integer total = firstResult + secondResult;
```

Prefer declaring the relationship:

```
CompletableFuture<Integer> totalFuture =
        firstFuture.thenCombine(
                secondFuture,
                Integer::sum
        );
```

This allows the completion pipeline to proceed automatically.

---

26. A Complete Pipeline

```
CompletableFuture<Void> pipeline =
        CompletableFuture
                .supplyAsync(() -> "User Data")
                .thenApply(data ->
                        data + " processed"
                )
                .thenAccept(result ->
                        System.out.println(result)
                );
```

Flow:

```
Load data
    ↓
Transform data
    ↓
Consume result
```

The submitting thread does not need to block between the stages.

Important CompletableFuture Design Points

27. Asynchronous Does Not Mean Non-Blocking Work

This task is asynchronous:

```
CompletableFuture.supplyAsync(() -> {
    return blockingDatabaseCall();
});
```

However, the worker thread executing it may still block during the database call.

```
Asynchronous
    → caller does not execute the work directly
```

```
Non-blocking
    → executing thread is not parked waiting for an operation
```

These are different ideas.

28. Be Careful With Blocking Work in the Common Pool

The common ForkJoin pool is shared by different APIs and application components.

Long blocking operations can occupy its workers and reduce progress for unrelated tasks.

For blocking I/O, consider:

- a dedicated executor
- an appropriately bounded I/O pool
- virtual threads

29. Do Not Ignore the Final Stage

In a short `main()` method, the JVM may terminate before asynchronous work visibly finishes.

For demonstrations, wait for the final stage:

```
pipeline.join();
```

In server applications, the application lifecycle normally remains active independently.

Part 3: ForkJoinPool

30. The Problem ForkJoinPool Solves

A normal thread pool works well when independent tasks are submitted to a shared executor.

```
Task queue
    ↓
Worker takes task
    ↓
Worker executes task
```

Some computations have a recursive structure:

```
Large problem
    ↓
Split into smaller problems
    ↓
Each smaller problem splits again
    ↓
Combine all partial results
```

Examples include:

- summing a large array
- merge sort
- recursive search
- image processing
- tree traversal
- mathematical divide-and-conquer algorithms

`ForkJoinPool` is designed for this pattern.

---

31. Divide-and-Conquer Parallelism

The basic strategy is:

```
1. Check whether the problem is small enough.
2. If yes, solve it directly.
3. Otherwise, split it into subtasks.
4. Execute subtasks in parallel.
5. Combine their results.
```

This requires a threshold.

```
Problem size <= threshold
    → solve directly

Problem size > threshold
    → split again
```

Without a threshold, recursive splitting would continue unnecessarily and create excessive overhead.

---

32. Meaning of Fork and Join

Fork

Forking schedules a subtask for asynchronous execution.

```
leftTask.fork();
```

Join

Joining waits for a subtask to finish and obtains its result.

```
Integer leftResult = leftTask.join();
```

```
Fork
    → allow a subtask to execute separately

Join
    → wait for and combine its result
```

Work Stealing

33. Why a Shared Queue Can Be Inefficient

In a divide-and-conquer computation, different workers may generate different amounts of work.

```
Worker A → many pending subtasks
Worker B → finishes early
```

Leaving Worker B idle wastes available CPU.

34. Worker-Local Queues

ForkJoinPool workers maintain local work queues.

A worker generally processes tasks from its own queue.

```
Worker A → local queue A
Worker B → local queue B
Worker C → local queue C
```

When a worker runs out of work, it looks for work in another worker's queue.

35. Work Stealing

```
Worker B becomes idle
        ↓
Worker B checks another worker's queue
        ↓
Worker B steals an available task
        ↓
Both workers continue processing
```

This is called **work stealing**.

It helps balance irregular recursive workloads without requiring every worker to compete constantly for one central queue.

36. Conceptual ForkJoinPool Flow

```
Main task submitted
        ↓
Worker starts computation
        ↓
Task splits into subtasks
        ↓
Subtasks enter worker-local queues
        ↓
Idle workers steal available subtasks
        ↓
Small tasks are solved directly
        ↓
Partial results are joined
        ↓
Final result is returned
```

ForkJoinTask Types

37. `RecursiveTask<V>`

Use `RecursiveTask<V>` when the computation returns a result.

```
class SumTask extends RecursiveTask<Integer> {

    @Override
    protected Integer compute() {
        return 10;
    }
}
```

---

38. `RecursiveAction`

Use `RecursiveAction` when the computation does not return a result.

```
class PrintTask extends RecursiveAction {

    @Override
```

```
    protected void compute() {
        System.out.println("Processing");
    }
}
```


Comparison

Type	Result
`RecursiveTask<V>`	Returns a value
`RecursiveAction`	Returns no value

Both extend `ForkJoinTask`.

ForkJoin Example

39. Sum an Array in Parallel

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

class SumTask extends RecursiveTask<Integer> {

    private static final int THRESHOLD = 2;

    private final int[] numbers;
    private final int start;
    private final int end;

    SumTask(int[] numbers, int start, int end) {
        this.numbers = numbers;
        this.start = start;
        this.end = end;
    }
```

```
    @Override
    protected Integer compute() {

        int length = end - start;

        if (length <= THRESHOLD) {
            return calculateDirectly();
        }

        int middle = start + length / 2;

        SumTask leftTask =
                new SumTask(
                        numbers,
                        start,
                        middle
                );

        SumTask rightTask =
                new SumTask(
                        numbers,
                        middle,
                        end
                );

        leftTask.fork();

        int rightResult = rightTask.compute();
        int leftResult = leftTask.join();

        return leftResult + rightResult;
    }

    private int calculateDirectly() {

        int sum = 0;
```

```
        for (int i = start; i < end; i++) {
            sum += numbers[i];
        }

        return sum;
    }
}

public class ForkJoinDemo {

    public static void main(String[] args) {

        int[] numbers = {
                1, 2, 3, 4,
                5, 6, 7, 8
        };

        ForkJoinPool pool = new ForkJoinPool();

        SumTask task =
                new SumTask(
                        numbers,
                        0,
                        numbers.length
                );

        int result = pool.invoke(task);

        System.out.println("Final sum: " + result);

        pool.shutdown();
    }
}
```

Output:

```
Final sum: 36
```

40. Understanding the Base Case

```java
if (length <= THRESHOLD) {
    return calculateDirectly();
}
```

The base case prevents further splitting when the problem becomes small.

```
Large range
    → split

Small range
    → calculate directly
```

The threshold is a performance decision.

A threshold that is too small creates too many tasks.

A threshold that is too large reduces parallelism.

41. The Standard Fork-Join Pattern

```java
leftTask.fork();

int rightResult = rightTask.compute();
int leftResult = leftTask.join();

return leftResult + rightResult;
```

The current worker:

1. forks one subtask
2. directly computes the other subtask

3. joins the forked task
4. combines both results

This is generally better than immediately forking both tasks and then waiting.

```
Fork one side
    +
Compute one side locally
    +
Join the forked side
```

It keeps the current worker productive and reduces unnecessary scheduling overhead.

---

42. `invoke()`, `fork()` and `join()`

Method	Purpose
`pool.invoke(task)`	Submit a top-level task and wait for its result
`task.fork()`	Schedule a subtask asynchronously
`task.join()`	Wait for a subtask and obtain its result
`task.compute()`	Execute the task logic directly in the current worker

---

43. The Common ForkJoinPool

Java maintains a shared pool:

```
ForkJoinPool.commonPool()
```

It is used by APIs such as:

- many `CompletableFuture` async methods
- parallel streams
- `ForkJoinTask.fork()` when called outside another ForkJoinPool

Because it is shared, long-running blocking tasks should not occupy it carelessly.

---

44. `newWorkStealingPool()`

A work-stealing executor can also be created through:

```
ExecutorService executor =
        Executors.newWorkStealingPool();
```

This creates a ForkJoinPool-oriented executor.

A target parallelism can be supplied:

```
ExecutorService executor =
        Executors.newWorkStealingPool(4);
```

It is useful for many small independent or recursively generated tasks, but it does not guarantee task execution order.

---

45. When ForkJoinPool Works Well

ForkJoinPool is best suited to workloads that are:

- CPU-bound
- divisible into independent subtasks
- recursive or naturally decomposable
- composed of many relatively small computations
- capable of combining partial results

Examples:

- recursive algorithms
- array computations
- tree processing
- parallel transformations
- parallel streams

---

46. When ForkJoinPool Is Not the Best Default

Avoid treating ForkJoinPool as a universal executor.

It is usually not the best choice when tasks:

- spend most of their time waiting for network I/O
- block on database calls
- hold locks for long durations
- depend heavily on task ordering
- cannot be divided into independent computations

Blocking a worker can reduce the pool's ability to process other tasks.

For blocking workloads, a dedicated executor or virtual threads are often more appropriate.

Part 4: ThreadLocal

47. What Problem Does ThreadLocal Solve?

A `ThreadLocal<T>` associates a separate value with each thread.

```
One ThreadLocal object
        ↓
Different value for each thread
```

Example:

```
Thread 1 → userName = "Harshita"
Thread 2 → userName = "Aditya"
```

Both threads access the same `ThreadLocal` variable, but each sees its own stored value.

---

48. Basic ThreadLocal Example

```
public class ThreadLocalDemo {

    private static final ThreadLocal<String> USER_NAME =
            new ThreadLocal<>();

    public static void main(String[] args)
            throws InterruptedException {

        Runnable firstTask = () -> {
            USER_NAME.set("Harshita");

            try {
                System.out.println(
                        Thread.currentThread().getName() +
                        " -> " +
                        USER_NAME.get()
                );
            } finally {
                USER_NAME.remove();
            }
        };

        Runnable secondTask = () -> {
            USER_NAME.set("Aditya");

            try {
                System.out.println(
                        Thread.currentThread().getName() +
                        " -> " +
                        USER_NAME.get()
                );
            } finally {
                USER_NAME.remove();
            }
        };
```

```
        Thread firstThread =
                new Thread(firstTask, "Thread-1");

        Thread secondThread =
                new Thread(secondTask, "Thread-2");

        firstThread.start();
        secondThread.start();

        firstThread.join();
        secondThread.join();
    }
}
```

Possible output:

```
Thread-1 -> Harshita
Thread-2 -> Aditya
```

---

49. Why Not Always Use a Local Variable?

A normal local variable is ideal when the value is only needed inside one method or can be passed explicitly.

```
void processRequest(String userName) {
    validate(userName);
    save(userName);
}
```

`ThreadLocal` becomes useful when contextual information must be available across several method calls without being passed through every method.

```
Controller
    ↓
```

```
Service
    ↓
Repository
```

Possible examples include:

- request identifiers
- tracing information
- security context
- transaction context
- tenant information

Use it carefully because it creates hidden data flow.

---

50. ThreadLocal and Thread Pools

Worker threads in a pool are reused.

```
Task A uses worker 1
        ↓
Task A completes
        ↓
Task B later uses worker 1
```

If Task A leaves a ThreadLocal value behind, Task B may observe stale data.

Always clean up values in a `finally` block:

```java
CONTEXT.set(value);

try {
    performWork();
} finally {
    CONTEXT.remove();
}
```

---

51. ThreadLocal and Virtual Threads

Virtual threads support ThreadLocal values.

However, a program may create a very large number of virtual threads. Large or expensive ThreadLocal values can therefore create significant memory consumption.

A virtual thread should normally represent one task and should not be pooled merely to reuse ThreadLocal data.

Part 5: Virtual Threads

52. Platform Threads

Traditional Java threads are called **platform threads**.

They are typically mapped closely to operating-system threads.

```
Java platform thread
        ↓
Operating-system thread
        ↓
CPU
```

Operating-system threads are relatively expensive resources.

A server that creates one platform thread for every concurrent blocking request may eventually face:

- high memory consumption
- thread-creation overhead
- excessive context switching
- operating-system thread limits

---

53. Why Blocking Applications Need Many Threads

Consider a server request:

```
Receive request
    ↓
Call database
    ↓
Wait for database
    ↓
Call another API
    ↓
Wait for response
    ↓
Return result
```

The thread may spend most of its lifetime waiting rather than using the CPU.

With platform threads, high concurrency requires many operating-system threads merely to represent all those waiting tasks.

Virtual threads reduce the cost of representing this concurrency.

---

54. What Is a Virtual Thread?

A virtual thread is a lightweight Java thread scheduled by the Java runtime rather than directly by the operating system.

```
Virtual threads
      ↓
JVM scheduler
      ↓
Carrier platform threads
      ↓
Operating system and CPU
```

Many virtual threads can be scheduled over a much smaller number of platform threads.

The platform thread temporarily running a virtual thread is called its **carrier thread**.

---

55. Mounting and Unmounting

When a virtual thread is ready to execute, the JVM mounts it on a carrier thread.

```
Virtual thread ready
        ↓
Mounted on carrier
        ↓
Executes Java code
```

When it reaches a supported blocking operation, such as waiting for I/O, the runtime can normally unmount it.

```
Virtual thread waits for I/O
        ↓
Unmounted from carrier
        ↓
Carrier becomes available
        ↓
Another virtual thread can run
```

When the operation becomes ready, the virtual thread can later resume on the same or a different carrier.

Application code still sees the virtual thread:

```
Thread.currentThread()
```

It does not receive the carrier thread.

---

56. Virtual Threads Do Not Create More CPU

Virtual threads improve scalability for tasks that spend significant time waiting.

They do not increase the number of CPU cores.

```
CPU-bound task
    → limited by available CPU

Blocking task
    → virtual threads can improve concurrency
```

Creating thousands of virtual threads for a CPU-intensive calculation does not make the CPU execute thousands of calculations simultaneously.

For CPU-bound divide-and-conquer work, ForkJoinPool is generally more suitable.

---

57. Java Version

Virtual threads became a permanent Java feature in Java 21.

The following APIs can be used without enabling preview features in Java 21 and later:

```
Thread.ofVirtual()
Thread.startVirtualThread(...)
Executors.newVirtualThreadPerTaskExecutor()
```

Creating Virtual Threads

58. `Thread.startVirtualThread()`

```
public class VirtualThreadDemo {

    public static void main(String[] args)
            throws InterruptedException {

        Thread thread =
                Thread.startVirtualThread(() -> {
                    System.out.println(
                            "Running in: " +
                            Thread.currentThread()
                    );
                });

        thread.join();
```

```
    }
}
```

This creates and starts a virtual thread.

It is equivalent to:

```
Thread.ofVirtual().start(task);
```

59. Virtual Thread Builder

```
Thread thread =
        Thread.ofVirtual()
                .name("order-task")
                .start(() -> {
                    System.out.println(
                            Thread.currentThread()
                    );
                });
```

An unstarted virtual thread can also be created:

```
Thread thread =
        Thread.ofVirtual()
                .name("order-task")
                .unstarted(task);

thread.start();
```

60. Check Whether a Thread Is Virtual

```
boolean virtual =
        Thread.currentThread().isVirtual();
```

Example:

```
Thread.startVirtualThread(() -> {
    System.out.println(
            Thread.currentThread().isVirtual()
    );
});
```

Output:

```
true
```

Virtual-Thread-Per-Task Executor

61. Creating the Executor

```
ExecutorService executor =
        Executors.newVirtualThreadPerTaskExecutor();
```

This executor starts a new virtual thread for each submitted task.

```
Task 1 → Virtual Thread 1
Task 2 → Virtual Thread 2
Task 3 → Virtual Thread 3
```

It is not a traditional thread pool.

Virtual threads are cheap enough that each task can receive its own thread. The executor does not reuse virtual threads after tasks finish.

---

62. Basic Executor Example

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class VirtualThreadExecutorDemo {

    public static void main(String[] args) {

        try (ExecutorService executor =
                     Executors
                             .newVirtualThreadPerTaskExecutor
()) {

            for (int i = 1; i <= 5; i++) {
                int taskId = i;

                executor.submit(() -> {
                    System.out.println(
                            "Task " + taskId +
                            " executed by " +
                            Thread.currentThread()
                    );
                });
            }
        }
    }
}
```

The try-with-resources block closes the executor and waits for submitted tasks to finish.

63. Blocking I/O Example

```
import java.time.Duration;
import java.util.concurrent.ExecutorService;
```

```
import java.util.concurrent.Executors;

public class VirtualThreadIoDemo {

    public static void main(String[] args) {

        try (ExecutorService executor =
                     Executors
                             .newVirtualThreadPerTaskExecutor
()) {

            for (int i = 1; i <= 10_000; i++) {
                int taskId = i;

                executor.submit(() -> {
                    try {
                        Thread.sleep(
                                Duration.ofSeconds(1)
                        );

                        System.out.println(
                                "Completed task " + taskId
                        );

                    } catch (InterruptedException exception)
{
                        Thread.currentThread()
                                .interrupt();
                    }
                });
            }
        }
    }
}
```

Sleeping represents a task waiting for an external operation.

While a virtual thread is waiting, its carrier can normally execute another virtual thread.

When to Use Virtual Threads

64. Suitable Workloads

Virtual threads work especially well for:

- web requests
- REST API calls
- database operations
- file and network I/O
- blocking queues
- message processing
- request-per-thread server designs
- workloads with many concurrent waiting tasks

They allow straightforward blocking code to scale without requiring every operation to be expressed as a callback chain.

65. Less Suitable Workloads

Virtual threads do not provide a major advantage when work is:

- long-running and CPU-intensive
- dominated by calculations rather than waiting
- limited by a small downstream resource
- already naturally handled as data parallelism

Examples:

- video encoding
- image rendering

- encryption
- large numerical computations
- recursive array processing

For CPU-bound tasks, limit concurrency close to available processing capacity.

Virtual Threads Are Not a License for Unlimited Work

66. Threads May Be Cheap, Dependencies Are Not

Even when virtual threads are inexpensive, the resources they use may remain limited.

Examples:

- database connection pool
- external API rate limit
- memory
- file descriptors
- sockets
- downstream service capacity

Creating 100,000 virtual threads does not create 100,000 database connections safely.

Concurrency should be limited around the scarce resource.

```
Semaphore databaseLimit =
        new Semaphore(50);
```

The purpose of virtual threads is to make waiting threads cheap, not to remove every system limit.

---

67. Do Not Pool Virtual Threads

With platform threads, pooling avoids expensive thread creation.

With virtual threads, the intended model is:

```
One task
    ↓
One new virtual thread
    ↓
Thread terminates with task
```

Do not create a small fixed pool of virtual threads to imitate a platform-thread pool.

Use:

```
Executors.newVirtualThreadPerTaskExecutor();
```

Limits should normally be applied to the constrained resource rather than by pooling virtual threads.

---

68. Virtual Threads and `synchronized`

Modern JDKs have reduced important sources of virtual-thread pinning, including monitor-related pinning improvements introduced after virtual threads first became permanent.

However, long blocking operations while executing native or foreign code can still occupy a carrier thread.

The practical rule remains:

- keep critical sections small
- do not hold locks while performing long external I/O
- measure real applications rather than assuming every blocking operation scales perfectly

Pinning does not make the code incorrect, but excessive carrier blocking can reduce scalability.

69. Virtual Threads and ThreadLocal

Virtual threads support ThreadLocal variables.

```
private static final ThreadLocal<String> REQUEST_ID =
        new ThreadLocal<>();
```

Because applications may create very many virtual threads:

- avoid large ThreadLocal objects
- remove values when no longer needed
- do not use ThreadLocal as a resource pool
- prefer explicit parameters when practical

Each virtual thread should be treated as an independent task context.

Choosing the Correct Tool

70. Future vs CompletableFuture

Requirement	Better choice
Track one task's result	`Future`
Wait or cancel one task	`Future`
Build a chain of dependent operations	`CompletableFuture`
Transform a result automatically	`CompletableFuture`
Combine independent async results	`CompletableFuture`
Handle a completion pipeline declaratively	`CompletableFuture`

71. ForkJoinPool vs Virtual Threads

ForkJoinPool	Virtual Threads
Primarily suited to CPU-bound parallel work	Primarily suited to high-concurrency blocking work
Splits computation into subtasks	Gives each task its own lightweight thread
Uses work stealing	Uses JVM scheduling over carrier threads
Best for divide and conquer	Best for thread-per-request/task
Parallelism limited by useful CPU capacity	Concurrency may be much larger than CPU count
Avoid long blocking operations	Blocking is the primary use case

72. CompletableFuture vs Virtual Threads

These tools are not direct replacements for each other.

CompletableFuture

```
Represent work as completion stages
    ↓
Attach callbacks
    ↓
Compose results declaratively
```

Useful when:

- asynchronous pipelines are natural
- several independent results must be combined
- a framework already works with completion stages
- callback-style composition is desirable

Virtual Threads

```
Write normal sequential blocking code
    ↓
Run each task on a virtual thread
```

```
    ↓
JVM handles large concurrency
```

Useful when:

- the workflow is naturally sequential
- code makes blocking I/O calls
- readability of imperative code is important
- thread-per-request programming is preferred

Example using CompletableFuture:

```
CompletableFuture<String> response =
        CompletableFuture
                .supplyAsync(() -> fetchUser())
                .thenCombine(
                        CompletableFuture
                                .supplyAsync(
                                        () -> fetchOrders()
                                ),
                        (user, orders) ->
                                buildResponse(user, orders)
                );
```

Conceptual virtual-thread version:

```
String user = fetchUser();
String orders = fetchOrders();

String response =
        buildResponse(user, orders);
```

The second version is easier to read but performs the two calls sequentially unless separate virtual threads or structured concurrency are used.

Final Mental Model

73. `Future`

```
Submit one task
    ↓
Receive a placeholder
    ↓
Wait, check or cancel
```

74. `CompletableFuture`

```
Submit or represent a stage
    ↓
Transform and combine stages
    ↓
React to completion or failure
```

75. `ForkJoinPool`

```
Split CPU-bound problem
    ↓
Workers process subtasks
    ↓
Idle workers steal work
    ↓
Join partial results
```

76. Virtual Threads

```
Create one lightweight thread per task
    ↓
Blocking virtual threads release carriers when possible
    ↓
Large numbers of waiting tasks become practical
```

Quick Revision

Concept	Core purpose
`Future<T>`	Represents the eventual result of one task
`future.get()`	Blocks until a result is available
`future.isDone()`	Checks terminal completion without blocking
`future.cancel(true)`	Requests cancellation and possible interruption
`CompletableFuture<T>`	Represents a composable completion stage
`runAsync()`	Runs an action without a result
`supplyAsync()`	Runs a supplier that returns a result
`thenApply()`	Transforms a result
`thenAccept()`	Consumes a result
`thenRun()`	Runs an action after completion
`thenCombine()`	Combines two successful results
`exceptionally()`	Recovers from failure
`whenComplete()`	Observes success or failure
`ForkJoinPool`	Executes divide-and-conquer tasks
`fork()`	Schedules a subtask
`join()`	Waits for and retrieves a subtask result
Work stealing	Lets idle workers take tasks from busy workers
`RecursiveTask<V>`	Fork-join task that returns a result
`RecursiveAction`	Fork-join task without a result
`ThreadLocal<T>`	Stores a separate value for each thread
Platform thread	Java thread typically backed by an OS thread
Virtual thread	Lightweight JVM-scheduled Java thread
Carrier thread	Platform thread temporarily running a virtual thread
Virtual-thread-per-task executor	Creates one virtual thread for each task

Final Takeaway

These concurrency tools solve different layers of the problem:

```
Future
    → How can I represent a result that will arrive later?

CompletableFuture
    → How can I build a pipeline around asynchronous completion?

ForkJoinPool
    → How can I parallelise a divisible CPU-bound computation?

Virtual threads
    → How can I handle many concurrent blocking tasks
      without requiring the same number of OS threads?
```

The correct tool depends on the nature of the workload—not simply on which API looks more modern.